

Informe Trabajo de Grado

Segundo Informe IDI (Intelligent Database Interface): Diseño del Sistema



Estudiante:

Juan David Ramírez Torres

Docente:

Félix Cortes Aldana

ÍNDICE

Capítulo 2: Diseño del Sistema

- 2.1. Arquitectura General de la Solución
- 2.2. La Obligación Didáctica como Preocupación Transversal de Diseño
- 2.3. Contratos de API entre Módulos
- 2.4. Diseño de la Cadena de Verificación Tripartita y Política de Auto-Corrección Silenciosa
- 2.5. Enrutamiento de Consultas: Salvaguarda de Filtrado por Lista de Permitidos
- 2.6. Gestión de Sesiones: Modelo de Datos y Ciclo de Vida
- 2.7. Estrategia de Comunicación de Progreso
- 2.8. Selección y Justificación del Stack Tecnológico
- 2.9. Decisión Arquitectónica: De LoRA Hot-Swap a Instruction-Profile Hot-Swap
- 2.10. Diseño Multi-Base de Datos y Descubrimiento Dinámico
- 2.11. Conclusiones del Capítulo
- 2.12. Recomendaciones

CAPÍTULO 2: DISEÑO DEL SISTEMA

Este capítulo desarrolla el segundo objetivo específico (OE2): diseñar la arquitectura modular de IDI, especificando responsabilidades de componentes, protocolos de comunicación inter-agente, flujos de datos, mecanismos de gestión de sesiones, estrategias de comunicación de progreso y selección de stack tecnológico. Toma como punto de partida la especificación de requerimientos del Capítulo 1 y la traduce en decisiones arquitectónicas concretas, varias de las cuales ya fueron validadas empíricamente durante la fase temprana de implementación. Bajo el propósito actualizado del Capítulo 1 — IDI como compañero didáctico cuya capacidad de responder enseñando habilita, como consecuencia, el acceso ejecutivo a los datos —, el diseño trata la obligación didáctica no como una funcionalidad más sino como una preocupación transversal de arquitectura: una restricción que el contrato de cada agente debe satisfacer (Sección 2.2). El resultado alimenta directamente el desarrollo de la solución (OE3, Capítulo 3).

2.1. ARQUITECTURA GENERAL DE LA SOLUCIÓN

La Figura 2.1 presenta la vista general de la solución: un frontend React conversacional, un backend FastAPI que aloja al orquestador y a los siete agentes, y tres servicios locales de soporte — el servidor de inferencia llama.cpp (Qwen2.5-Coder-3B-Instruct en formato GGUF Q4_K_M, ~2 GB de VRAM), el almacén vectorial ChromaDB y la base SQLite de sesiones. Todos los componentes corren en la máquina del usuario, en coherencia con el principio rector local-first (Capítulo 1): ningún dato ni consulta sale del equipo, y el presupuesto de VRAM se mantiene por debajo de 3.5 GB con un perfil de especialización activo.

IDI — Vista general de la solución (local-first)

Todos los componentes corren en la máquina del usuario · presupuesto < 3.5 GB VRAM

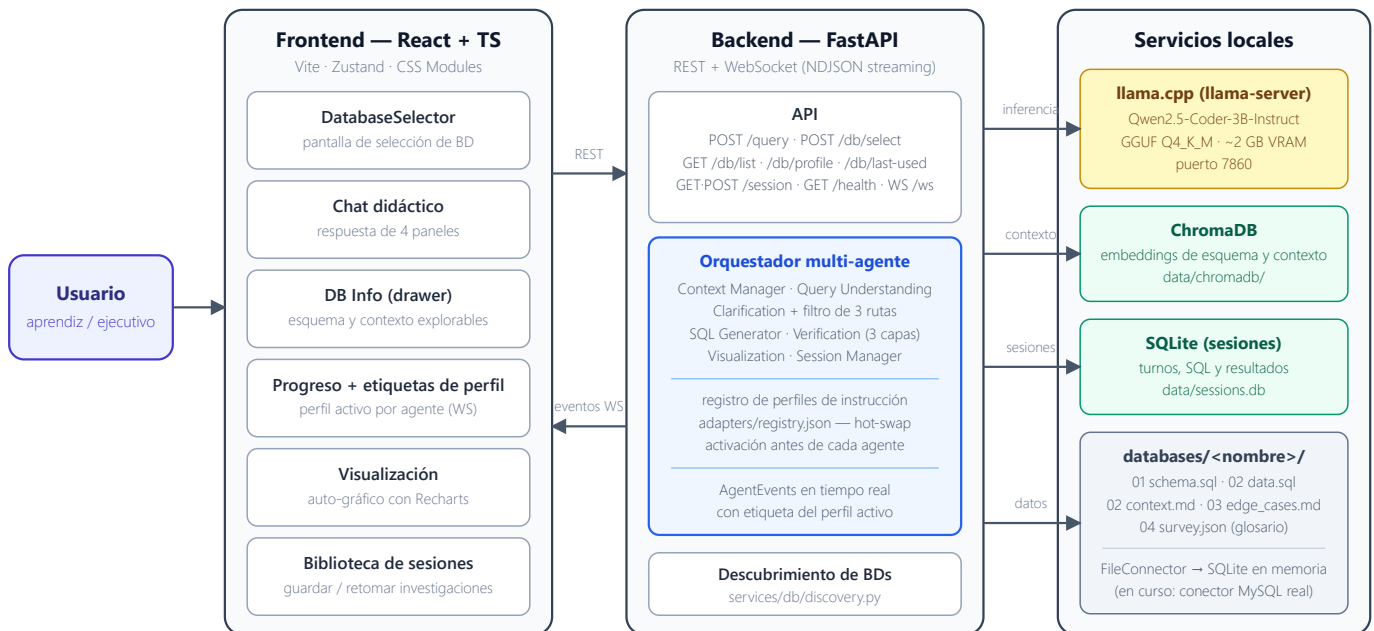


Figura 2.1 — Arquitectura general de la solución IDI. Fuente: elaboración propia a partir del repositorio (backend/app, frontend/, databases/).

- ▣ pantalla de selección de base de datos (DatabaseSelector) y vista general del chat con el drawer "DB Info" abierto.

El sistema orquesta siete agentes especializados — Context Manager, Query Understanding, SQL Generator, Verification, Visualization Engine, Session Manager y Multi-Agent Orchestrator — coordinados por un orquestador central que activa el perfil de instrucción correspondiente antes de invocar a cada agente. La Figura 2.2 muestra el recorrido de una consulta a través del pipeline, incluidas sus rutas de respuesta sin generación de SQL: las respuestas fuera del propósito de IDI (filtro de rutas, Sección 2.5), las respuestas de la base de datos seleccionada (respuesta directa anclada al **DBProfile** y pregunta de clarificación al usuario) y el ciclo de verificación (si el SQL no pasa las tres capas, se genera un nuevo **SqlCandidate** con un único reintento de reparación silenciosa). La arquitectura fue validada de forma incremental: el pipeline agéntico completo corre de extremo a extremo sobre **/query** desde la fase temprana de implementación (DB-less, alimentado por **FileConnector**).

Pipeline agéntico de IDI — orquestación de una consulta (/query)

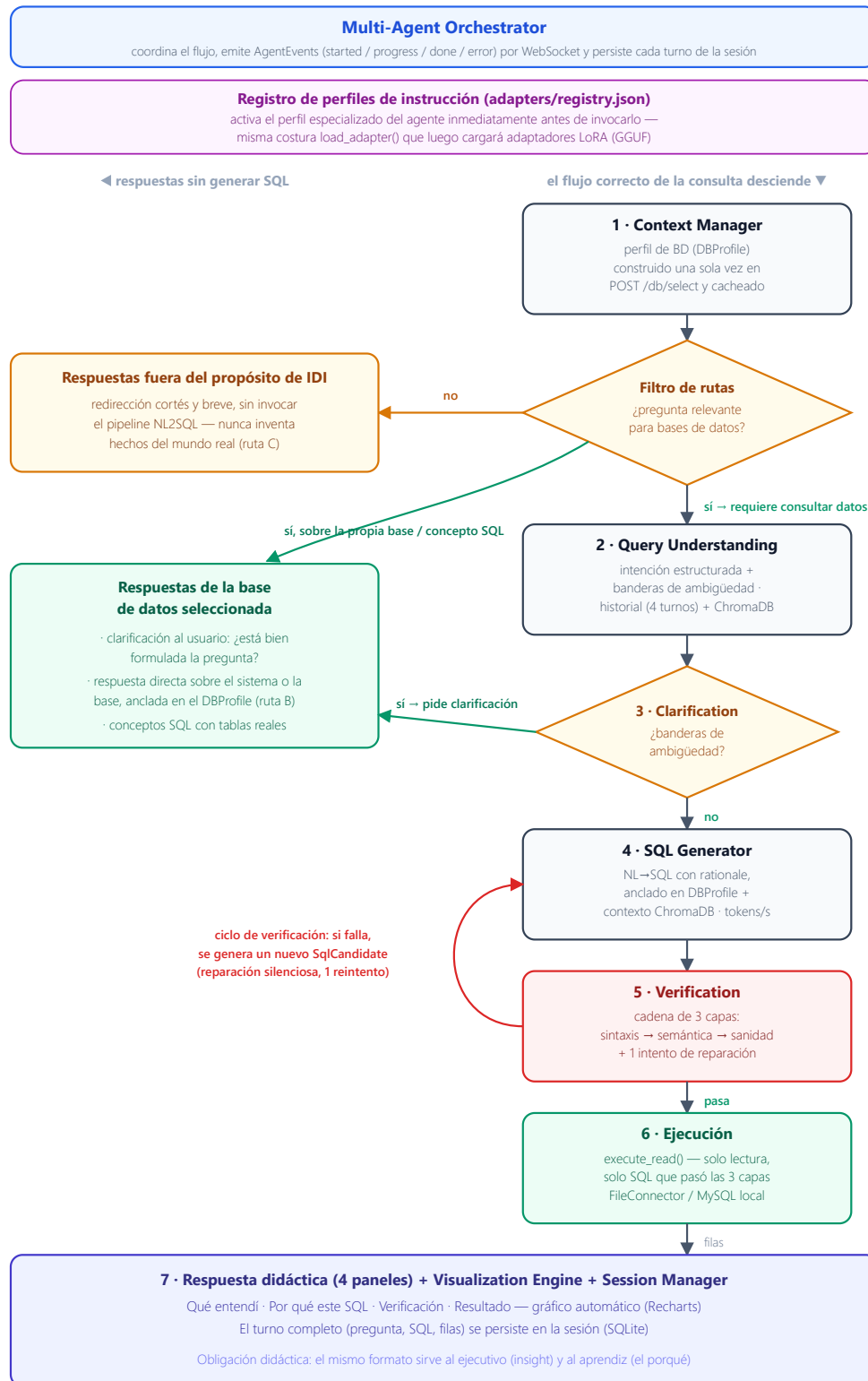


Figura 2.2 — Pipeline agéntico: el flujo correcto de la consulta desciende por la derecha; a la izquierda, las respuestas que no generan SQL. Fuente: backend/app/services/orchestrator.py (elaboración propia).

Agente	Responsabilidad	Estado de diseño
Context Manager Agent	Adquisición de contexto y glosario de negocio	Implementado (perfil construido una sola vez en <code>POST /db/select</code> y cacheado)
Query Understanding Agent	Parseo de intención y detección de ambigüedad	Implementado; endurecido con salvaguarda de filtrado (Sección 2.5)
SQL Generator Agent	Traducción NL→SQL	Implementado
Verification Agent	Verificación sintáctica/semántica/de sanidad	Implementado
Visualization Engine	Selección automática de gráfico	Implementado
Session Manager Agent	Persistencia y continuidad de sesiones	Implementado, con defecto conocido KI-1
Multi-Agent Orchestrator	Enrutamiento y ciclo de vida	Implementado, con activación de perfiles de instrucción

2.2. LA OBLIGACIÓN DIDÁCTICA COMO PREOCUPACIÓN TRANSVERSAL DE DISEÑO

El Capítulo 1 (§1.6) asignó a cada módulo un requerimiento didáctico transversal: además de ejecutar su función, cada agente debe exponer al usuario – aprendiz o ejecutivo – el razonamiento detrás de lo que hizo. Arquitectónicamente, esto no se diseñó como un octavo módulo "didáctico" sino como una restricción sobre los contratos de salida de los siete existentes: el mismo formato de respuesta sirve al ejecutivo (que lee el insight y actúa) y al aprendiz (que lee el porqué y aprende). Cuatro decisiones de diseño, ya visibles en el artefacto, materializan esta restricción:

1. La respuesta didáctica de 4 paneles como formato canónico de salida: toda respuesta expone qué entendió el sistema, qué SQL construyó y por qué, qué resultado obtuvo y cómo se visualiza – el vehículo de los requerimientos didácticos del SQL Generator y del Visualization Engine.

▣ una respuesta completa de 4 paneles en el chat, mostrando "Qué entendí / Por qué este SQL / Verificación / Resultado".

2. La trazabilidad del perfil activo: el orquestador incorpora la etiqueta del perfil de instrucción activo en el evento **"started"** de cada agente, y el frontend la muestra como etiqueta de perfil activo – el aprendiz puede ver qué habilidad está usando el modelo en cada fase del pipeline (requerimiento didáctico del Multi-Agent Orchestrator; escenario UC-07).
3. El perfil de base de datos como material de estudio: la decisión de exponer una pestaña "DB Info" (drawer de perfil de BD) responde a una constatación de diseño – el mismo contexto que el sistema construye para sí mismo (esquema introspectado, glosario de negocio, descripciones de tablas) es exactamente el material que un aprendiz necesita para orientarse en una base de datos desconocida. En lugar de mantener ese contexto como un artefacto interno del pipeline, se publicó en la interfaz: el usuario puede leer y explorar la base de datos activa antes o durante la conversación – semilla del mini-glosario navegable exigido al Context Manager Agent.

▣ pestaña/drawer "DB Info" mostrando el esquema de SoundWave con sus descripciones y glosario.

4. Las respuestas no-SQL también enseñan: bajo el enrutamiento de tres rutas (Sección 2.5), una pregunta conceptual ("¿qué es un JOIN?") no se responde en abstracto sino ilustrada con las tablas y columnas reales de la base conectada, y una pregunta sobre la propia base de datos ("¿qué tablas tienes?") se responde siempre desde los hechos actuales del **DBProfile** — la lección usa el terreno de práctica del estudiante, no un ejemplo genérico.

2.3. CONTRATOS DE API ENTRE MÓDULOS

La comunicación entre el frontend y el backend, y entre el orquestador y sus agentes, se rige por contratos tipados: todo mensaje que cruza una frontera de módulo es una instancia de un modelo Pydantic (backend) con su espejo TypeScript (frontend). Esta decisión responde a la recomendación heredada del Capítulo 1 (§1.13, OE2) — formalizar los contratos antes de probar la verificación y la orquestación en aislamiento — y al RNF de tipado estricto en fronteras de agente.

La superficie pública del backend consta de los siguientes endpoints:

Endpoint	Método	Entrada	Salida	Propósito
/health	GET	—	{status, llm_healthy}	Latido del backend y del servidor de inferencia
/db/list	GET	—	Lista de {db_name, display_name, description, has_survey}	Bases descubiertas dinámicamente en databases/
/db/last-used	GET	—	{db_name}	Atajo "usar la última base" (derivado del historial de sesiones)
/db/select	POST	{db_name}	DBProfile completo	Selección explícita: construye y cachea el perfil una sola vez
/db/profile	GET	—	DBProfile de la base activa	Alimenta el drawer "DB Info"
/query	POST	{message, session_id?}	Flujo NDJSON: $n \times \text{AgentEvent} + 1 \times \text{QueryResult}$ final	Punto de entrada del pipeline agéntico
/session	GET / POST	— / metadatos de sesión	Lista de sesiones / sesión creada	Biblioteca de sesiones
/session/{id}	GET	—	Sesión con sus turnos	Restauración de una investigación previa
/ws	WebSocket	—	AgentEvents en tiempo real	Progreso en vivo durante consultas largas

Los tres modelos que estructuran el flujo de una consulta son:

Modelo	Campos principales	Quién lo produce → quién lo consume
AgentEvent	session_id, agent, status (started/progress/done/error), message, payload (incluye adapter, el perfil activo)	Orquestador → frontend (stream NDJSON y WebSocket)
SqlCandidate	sql, rationale, generation_method	SQL Generator → Verification Agent
QueryResult	session_id, intent, sql, verify (reporte de las 3 capas), rows, row_count, teaching_summary, error	Orquestador → frontend (mensaje final del stream)

El contrato de `/query` merece una nota de diseño: la respuesta no es un objeto único sino un flujo NDJSON – una secuencia de `AgentEvents` que narra el avance del pipeline, cerrada por un único `QueryResult`. Esta forma de contrato es la que hace posible, con un solo endpoint, satisfacer simultáneamente el RNF de progreso informado (consultas de hasta 30 segundos) y la obligación didáctica de transparencia (el usuario ve qué agente trabaja y con qué perfil). FastAPI genera además la especificación OpenAPI de esta superficie de forma automática a partir de los mismos modelos Pydantic, de modo que la documentación del contrato no puede desalinearse de su implementación.

2.4. DISEÑO DE LA CADENA DE VERIFICACIÓN TRIPARTITA Y POLÍTICA DE AUTO-CORRECCIÓN SILENCIOSA

La verificación se diseñó como una cadena no negociable de tres capas – sintaxis, semántica, sanidad – descrita en `MASTERPLAN.md` §4 y representada en la Figura 2.3. La capa sintáctica combina el parseo con `sqlglot` y un `EXPLAIN` contra el motor real; si falla, las capas restantes se omiten y el SQL se bloquea de inmediato. La capa semántica exige que toda tabla y columna referenciada exista en el `DBProfile` – un ataque directo a la categoría de error dominante identificada en el Capítulo 1 (schema linking, 27–68% de los fallos). La capa de sanidad garantiza que solo consultas de lectura alcancen la ejecución: ningún DDL/DML (DROP, DELETE, UPDATE) puede atravesarla, materializando el principio fail-safe.

Cadena de verificación tripartita + auto-corrección silenciosa

Principio fail-safe: ningún SQL sin verificar llega a ejecutarse

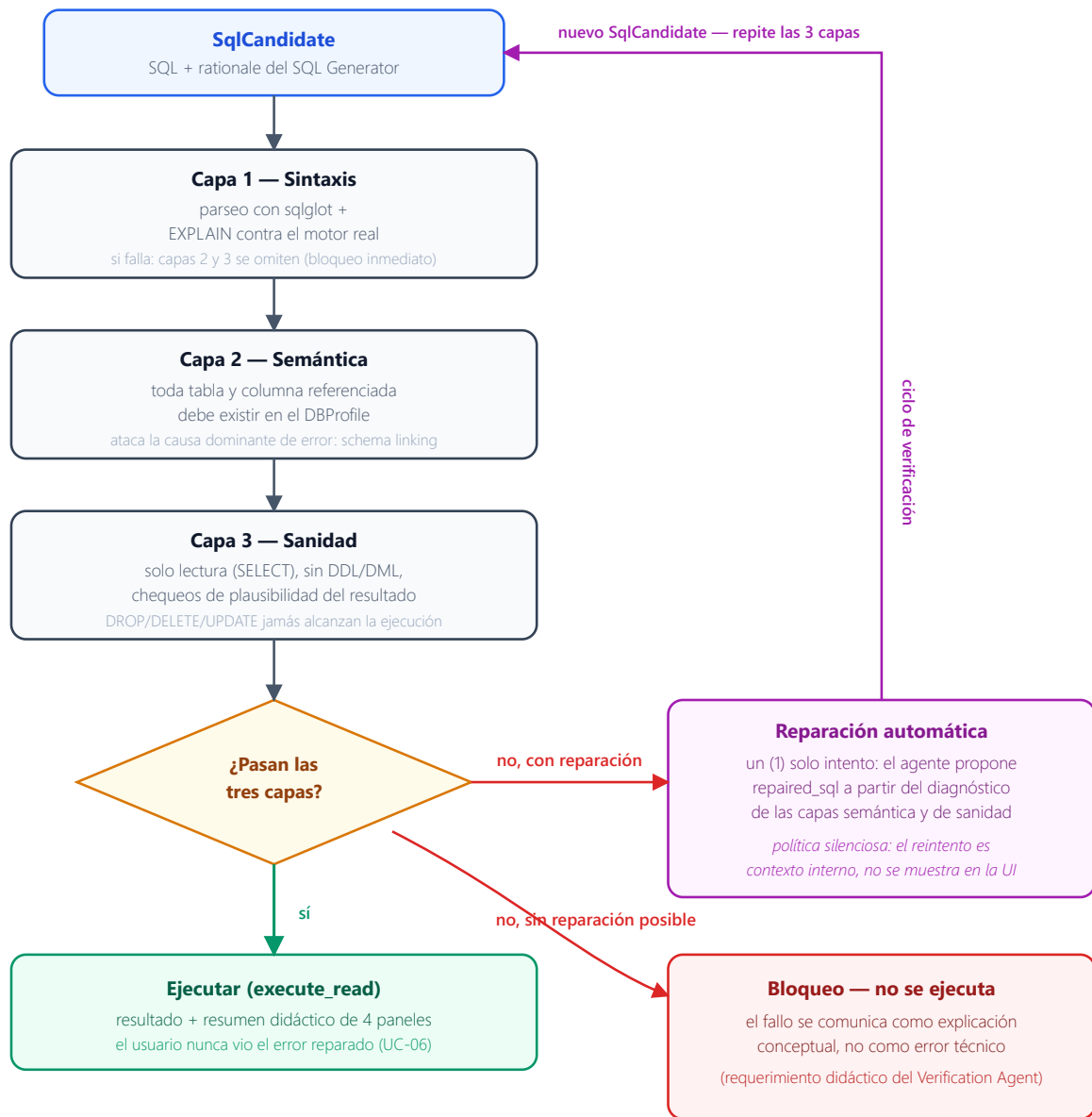


Figura 2.3 — Cadena de verificación y política de auto-corrección silenciosa. Fuente: backend/app/agents/verification.py y orchestrator.py (elaboración propia).

Una decisión de diseño fijada en la actualización del Capítulo 1 gobierna el comportamiento de esta cadena hacia el usuario: la auto-corrección es silenciosa (UC-06). Cuando la verificación falla pero el agente puede proponer una reparación, se ejecuta un único intento de reparación automática y el SQL reparado repite las tres capas completas; los fallos que el reintento logra resolver se consumen como contexto interno del pipeline — el usuario nunca ve un resultado erróneo ni el detalle del error corregido. Solo los fallos que no pueden resolverse automáticamente (o los bloqueos deliberados de seguridad) se comunican, y se comunican como explicación conceptual, no como mensaje de error técnico (requerimiento didáctico del Verification Agent, Capítulo 1 §1.6).

2.5. ENRUTAMIENTO DE CONSULTAS: SALVAGUARDA DE FILTRADO POR LISTA DE PERMITIDOS

Decisión de diseño posterior al Capítulo 1, surgida durante el endurecimiento del pipeline (commit [8cd337b](#), 2026-07-06) y candidata a documentarse como ADR: no toda entrada del usuario debe llegar al pipeline NL2SQL. Un modelo de lenguaje competente responde con gusto cualquier pregunta – sobre el clima, sobre sí mismo, sobre religión – y precisamente por eso necesita límites de alcance explícitos: sin ellos, el sistema quemaría un ciclo de generación de SQL en preguntas que ningún SQL puede responder, o peor, inventaría hechos del mundo real. El diseño distingue tres rutas de respuesta, representadas en la Figura 2.4:

- a) preguntas relacionadas con la base de datos activa o con conocimiento de SQL: siguen el pipeline completo (intención → SQL → verificación → resultado);
- b) preguntas sobre el sistema o sobre la base de datos seleccionada ("¿qué tablas tienes?", "¿qué puedes hacer?") y preguntas conceptuales de SQL ("¿qué es un JOIN?"): se responden por una ruta directa – las respuestas de la base de datos seleccionada –, siempre fundamentada en hechos actuales del **DBProfile** – nunca desde la memoria del modelo – e ilustrada con las tablas reales de la base conectada;
- c) preguntas no relevantes para bases de datos (personales o de conocimiento general): quedan fuera del propósito de IDI y se contestan con una redirección cortés y breve, sin invocar el pipeline NL2SQL y sin inventar jamás un hecho del mundo real (ni clima, ni precios, ni noticias).

Enrutamiento de tres rutas — salvaguarda por lista de permitidos (allowlist)

Ninguna pregunta genera SQL si no se reconoce como pertinente al dominio — el filtro falla de forma segura

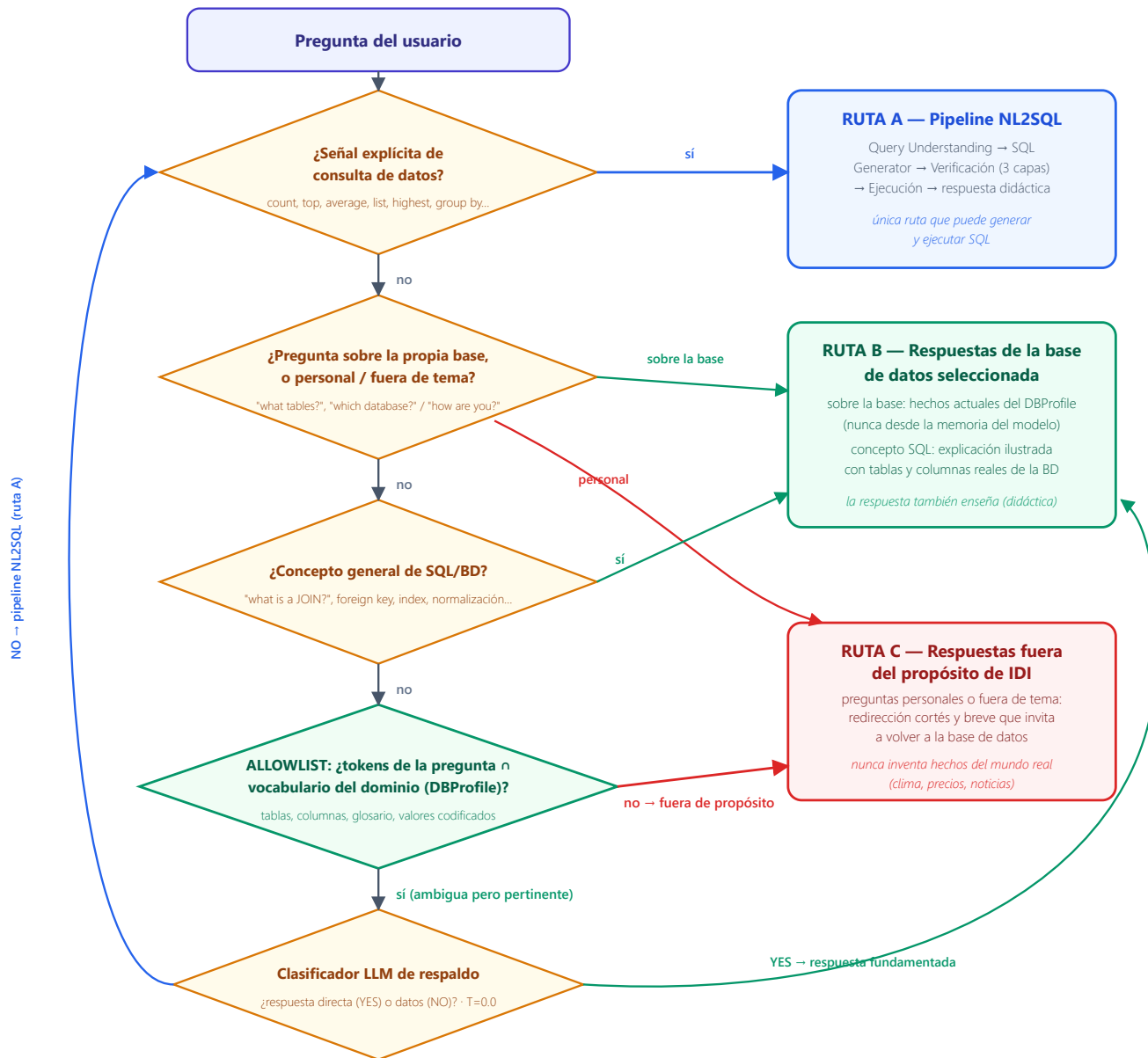


Figura 2.4 — Decisión de tres rutas del MetaQuestionFilter. Fuente: backend/app/agents/clarification.py, commit 8cd337b (elaboración propia).

La pertenencia a la ruta (a) se decide por lista de permitidos (allowlist) — intersección de los tokens de la pregunta con el vocabulario del dominio derivado del **DBProfile** (nombres de tablas y columnas, glosario, valores codificados), o señal de pregunta de conocimiento SQL — y no por una lista negra de patrones. La razón: una blocklist de formulaciones fuera de tema siempre queda un caso por detrás de las formulaciones imprevistas ("what's the weather" no atrapa "how is the weather"), mientras que la allowlist falla de forma segura — lo que no se reconoce como pertinente no genera SQL, en coherencia con el principio fail-safe del proyecto. Para la zona gris (preguntas que rozan el vocabulario del dominio pero no son claramente de datos), el diseño añade un clasificador LLM de respaldo con temperatura 0.0 que decide de forma binaria entre respuesta directa y pipeline.

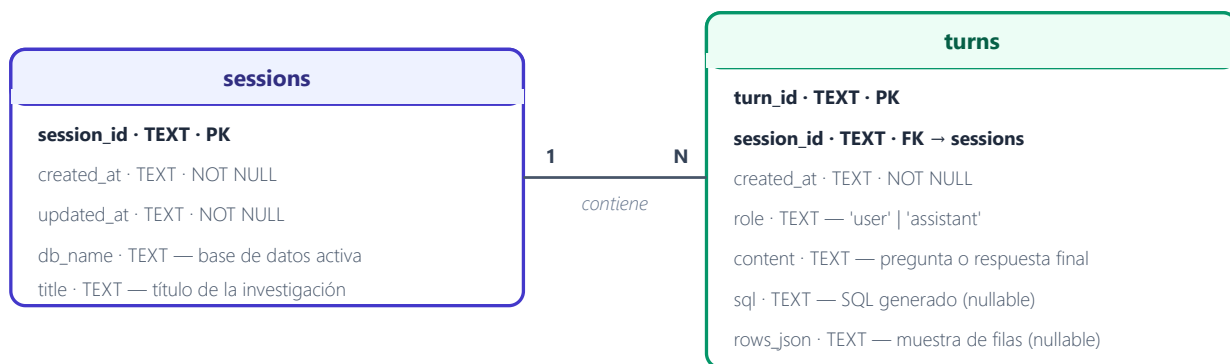
- ejemplo real de las tres rutas en el chat — una consulta de datos respondida con SQL, una pregunta sobre la base seleccionada respondida desde el perfil, y una pregunta fuera del propósito de IDI redirigida cortésmente.

2.6. GESTIÓN DE SESIONES: MODELO DE DATOS Y CICLO DE VIDA

La Figura 2.5 presenta el modelo entidad-relación de `data/sessions.db`: una entidad `sessions` (identificador, fechas de creación y actualización, base de datos activa y título de la investigación) que contiene N entidades `turns` — cada turno persiste el rol, el contenido, el SQL generado y una muestra en JSON de las filas obtenidas, de modo que los turnos del asistente guardan la lección completa y no solo el diálogo.

Modelo de datos de sesiones (`data/sessions.db` — SQLite)

memoria episódica: qué se conversó en cada sesión — una sesión contiene N turnos



- Índice: `idx_turns_session(session_id)` — recuperación eficiente de los turnos de una sesión.
- `sql` y `rows_json` solo se pueblan en turnos del asistente: una sesión restaurada puede reconstruir la lección completa.
- La base se crea (o auto-repara vacía) en el arranque del backend — cero servicios que instalar.

Figura 2.5 — Modelo entidad-relación de `data/sessions.db`. Fuente: `backend/app/services/memory/sessions.py` (elaboración propia).

El diseño soporta guardar, retomar, buscar y exportar sesiones investigativas (UC-03 del Capítulo 1), incluyendo la marcación de una sesión como "ruta de aprendizaje" exportable como material de estudio (requerimiento didáctico del Session Manager Agent). Cada turno persiste la pregunta del usuario y, para las respuestas del asistente, el resumen didáctico, el SQL generado y una muestra de las filas obtenidas — de modo que una sesión restaurada pueda reconstruir la lección completa, no solo el diálogo.

2.7. ESTRATEGIA DE COMUNICACIÓN DE PROGRESO

El diseño usa WebSockets para emitir `AgentEvents` de progreso en tiempo real, con el perfil de instrucción activo (`adapter`) incorporado en el evento `"started"` de cada agente — de modo que el frontend puede mostrar qué habilidad está usando el modelo en cada fase, sin requerir cambios adicionales en el store del frontend. Esta trazabilidad cumple una doble función fijada en el Capítulo 1: comunicación de progreso para el ejecutivo (RNF de

desempeño, consultas de hasta 30 segundos con progreso informado) y transparencia arquitectónica para el aprendizaje (requerimiento didáctico del Multi-Agent Orchestrator, escenario UC-07). Como decisión de experiencia de usuario frente a las consultas largas, el diseño no persigue estimar el tiempo restante: la espera se acompaña con contenido.

▣ barra de progreso del pipeline con las etiquetas de perfil activo por agente durante una consulta en curso.

2.8. SELECCIÓN Y JUSTIFICACIÓN DEL STACK TECNOLÓGICO

Capa	Tecnología	Justificación
Motor de inferencia	llama.cpp + Qwen2.5-Coder-3B-Instruct (Q4_K_M)	Validado en el sandbox (Capítulo 1, §1.11): ~2GB VRAM, 25–35 tok/s en GPU, funcional en CPU-only
Backend	FastAPI (Python)	Ver justificación detallada abajo
Frontend	React + TypeScript + Zustand, CSS Modules (sin Tailwind)	Decisión fijada al inicio del desarrollo (MASTERPLAN)
Persistencia de sesiones	SQLite (data/sessions.db)	Embebida en la biblioteca estándar de Python: cero servicios que instalar, auto-reparable (RNF de seguridad/offline)
Contexto semántico	ChromaDB (data/chromadb/)	Ver justificación detallada abajo

FastAPI se eligió como marco del backend por la convergencia de cuatro propiedades que el diseño de IDI exige simultáneamente: primero, su soporte asíncrono nativo es la base técnica del contrato de streaming de `/query` (Sección 2.3) y del canal WebSocket de progreso (Sección 2.7) — un framework síncrono habría obligado a simular el streaming o a bloquear el servidor durante los hasta 30 segundos de una consulta; segundo, su integración de primera clase con Pydantic convierte el RNF de tipado estricto en fronteras de agente en una propiedad verificada en tiempo de ejecución, y produce gratis la especificación OpenAPI de todos los contratos; tercero, al ser Python comparte lenguaje con todo el ecosistema de IA del proyecto (cliente de llama.cpp, ChromaDB, sqlglot), evitando una frontera de serialización adicional entre el backend y los agentes; y cuarto, su huella de recursos es compatible con el presupuesto de hardware de consumo, donde la GPU y la RAM deben reservarse para la inferencia.

ChromaDB se eligió como almacén del contexto semántico — los embeddings del esquema y de los documentos de contexto de dominio que el Context Manager ingesta al seleccionar una base, y que el Query Understanding y el SQL Generator recuperan para anclar sus decisiones al esquema real. Tres razones lo sostienen frente a las alternativas: opera embebido y persiste en un directorio local ([data/chromadb/](#)), sin ningún servidor adicional que administrar — coherente con local-first, donde una alternativa como pgvector exigiría operar un PostgreSQL y una solución en la nube (Pinecone) está descartada de plano por privacidad; integra la función de embeddings (SentenceTransformer) y la gestión de colecciones y metadatos en una sola pieza, donde una biblioteca de índices puros como FAISS habría requerido construir a mano la capa de persistencia y metadatos; y su almacén se auto-reconstruye si se borra (el Context Manager re-ingesta el contexto en la siguiente selección de base), lo que lo hace operacionalmente indestructible para un usuario no técnico. La división del trabajo de persistencia es deliberada: ChromaDB guarda el

conocimiento semántico (qué significa el esquema), SQLite guarda la memoria episódica (qué se conversó en cada sesión) – dos problemas distintos, dos herramientas mínimas.

2.9. DECISIÓN ARQUITECTÓNICA: DE LoRA HOT-SWAP A INSTRUCTION-PROFILE HOT-SWAP

Esta sección documenta una desviación explícita frente al Capítulo 1 y debe redactarse como un Architecture Decision Record (ADR), tal como recomienda el propio Capítulo 1 (§1.13, OE2).

El Capítulo 1 (§1.10) especificó adaptadores LoRA intercambiables en caliente como el mecanismo de especialización. Para avanzar rápidamente en el desarrollo, toda la lógica del backend se levantó – y sus pruebas se ejecutan – usando únicamente **perfiles de instrucción** (`backend/app/prompts/<agente>.md`), registrados en `adapters/registry.json` y activados por `backend/app/services/adapter_registry.py` inmediatamente antes de que cada agente se ejecute. El seam de `load_adapter()` es exactamente el mismo que más adelante cargará los adaptadores GGUF: cuando los LoRA estén disponibles, se conectan sin tocar la arquitectura.

El entrenamiento de los adaptadores LoRA quedó pendiente durante esa fase y actualmente se está llevando a cabo. Las corridas recientes de la batería de casos límite muestran que la especialización por perfiles de instrucción ya resuelve la mayor parte de la taxonomía, lo que acota el objetivo del entrenamiento: cerrar los huecos que el prompting no cierra, no construir la capacidad desde cero.

2.10. DISEÑO MULTI-BASE DE DATOS Y DESCUBRIMIENTO DINÁMICO

Otra desviación de diseño no anticipada en el Capítulo 1, que conviene contar como una evolución en dos pasos. En el sandbox inicial, el contexto de dominio era deliberadamente hardcodeado: el glosario de negocio y la descripción del esquema vivían dentro del código (`context_manager.py`), lo justo para levantar el pipeline y comprobar que la arquitectura funcionaba sobre una única base fija (SoundWave). En el diseño actual, ese contexto se externalizó a archivos que describen la base de datos, provistos por el usuario y explorados localmente por el sistema: cada carpeta bajo `databases/` contiene el esquema SQL, la narrativa de dominio y una encuesta de glosario (convención `NN_<db>_survey.json`), y `backend/app/services/db/discovery.py` la descubre dinámicamente – dejar caer una carpeta nueva basta para tener un nuevo terreno de práctica, sin cambios de código. En el mismo movimiento, `SoundwaveFileConnector` se generalizó a `FileConnector` (parametrizado por `db_name`).

Esta modalidad basada en archivos es la adecuada para la fase de diseño y comprobación: permite validar el pipeline completo sin depender de un servidor de base de datos. El mismo diseño contempla, como paso siguiente ya en curso, la conexión a una base SQL corriendo localmente (MySQL), sobre el mismo contrato de solo lectura (`execute_read()`) que hoy cumple el `FileConnector` – de modo que el sistema pueda, más adelante, conectarse por medio de API a cualquier base de datos relacional.

Bajo el propósito didáctico actualizado del Capítulo 1, esta decisión adquiere una justificación adicional: un entorno de aprendizaje gana valor si el estudiante puede practicar sobre bases de datos distintas.

2.11. CONCLUSIONES DEL CAPÍTULO

Primera (aporta a OE2) – los guardrails de alcance son una pieza de diseño de primera clase, no un parche: un modelo de lenguaje tan capaz como los actuales responde con fluidez cualquier pregunta – y precisamente por eso, un sistema NL2SQL serio debe decidir por diseño qué preguntas *no* debe intentar responder. La salvaguarda de tres rutas (Sección 2.5) demostró que el mecanismo correcto es la lista de permitidos anclada al vocabulario real de la base conectada, y no una lista negra de formulaciones prohibidas: la allowlist falla de forma segura (lo no reconocido como pertinente jamás genera SQL ni hechos inventados), mientras que una blocklist siempre queda una formulación por detrás. El guardrail, además, no empobrece el sistema sino que lo enriquece didácticamente: la pregunta desviada recibe una respuesta fundamentada en el perfil real de la base o una redirección honesta, nunca una alucinación.

Segunda (aporta a OE2) – la división del problema NL2SQL en agentes especializados orquestados, con la verificación como cadena obligatoria de tres capas, es la decisión que más precisión aporta por unidad de complejidad: cada agente resuelve una sub-tarea acotada con un perfil de instrucción a su medida, y ningún SQL alcanza la ejecución sin atravesar sintaxis, semántica y sanidad (Sección 2.4). La evidencia más significativa es que este resultado se obtuvo **sin entrenar ningún adaptador LoRA**: la especialización por simples perfiles de instrucción por agente (Sección 2.9), intercambiados por la misma costura `load_adapter()` que luego cargará los LoRA reales, bastó para que el pipeline resolviera correctamente casos límite documentados (manejo de NULL, vigencia temporal, resultado vacío honesto) – lo que valida tanto la arquitectura de especialización como la decisión de diferir el entrenamiento – hoy en curso – hasta tener la arquitectura probada.

Tercera (aporta a OE2) – hacer que el sistema construya su propio contexto a partir de los archivos de la base de datos (esquema SQL, narrativa de dominio, encuesta de glosario) resultó doblemente rentable: como diseño, el **DBProfile** construido una sola vez y cacheado ancla a todos los agentes a los hechos reales del esquema (es la razón por la que el sistema responde 0 filas ante un artista inexistente en lugar de inventarlo); y como didáctica, ese mismo contexto se publicó al usuario en la pestaña "DB Info", convirtiendo el artefacto interno del pipeline en material de estudio explorable – la decisión de diseño y la obligación didáctica se satisficieron con una sola pieza (Secciones 2.2 y 2.10).

Cuarta (aporta a OE2) – la validación empírica acompañó a cada decisión de diseño en lugar de esperarla al final: el pipeline completo corre de extremo a extremo desde la fase temprana de implementación, y las corridas más recientes de la batería de casos límite (EC-01...EC-08) muestran una mejora sostenida frente a las primeras mediciones, con los huecos restantes concentrados en frentes localizados – el mapeo de valores codificados (la cara concreta del schema linking, categoría de error dominante según la literatura del Capítulo 1) y la calibración del detector de ambigüedad. Ninguno de esos huecos cuestiona la arquitectura: todos son calibrables por perfil de instrucción o por el entrenamiento LoRA en curso, que así tiene un objetivo empíricamente delimitado. El detalle de esa evidencia y su análisis pertenecen a los Capítulos 3 y 4.

2.12. RECOMENDACIONES

Primera (OE2) – dejar por escrito, en un documento corto por cada decisión (un ADR), las tres decisiones de diseño que cambiaron frente a la propuesta original: la salvaguarda de tres rutas (Sección 2.5), la especialización por perfiles de instrucción (Sección 2.9) y el diseño multi-base de datos (Sección 2.10). El objetivo es simple: que el jurado pueda comparar lo que se propuso en el CADE con lo que finalmente se construyó, sin tener que reconstruir esa historia desde los commits del repositorio.

Segunda (OE2 → OE3) – completar el entrenamiento LoRA en curso (Sección 2.9) con el objetivo que la evidencia de los benchmarks ya delimitó: cerrar los huecos que la especialización por perfiles de instrucción no cierra – en particular el mapeo de valores codificados y la calibración del detector de ambigüedad –, sin rediseñar la arquitectura.

Tercera (OE2 → OE4) – cerrar el defecto conocido KI-1 (restauración de sesiones, Sección 2.6, Figura 2.5) antes de ejecutar la evaluación del Capítulo 4, de modo que una sesión restaurada reconstruya la lección completa – respuestas del asistente incluidas – y no solo las preguntas del usuario.

Cuarta (OE2) – implementar las frases didácticas sobre bases de datos durante la espera (Sección 2.7) y seguir midiendo la latencia de generación durante la evaluación: la espera deja de ser un costo si también enseña, pero el RNF de desempeño del Capítulo 1 sigue vigente como objetivo de calibración.

Quinta (OE2) – incluir como anexo del informe final la especificación OpenAPI exportada del backend (`/openapi.json`), de modo que los contratos de la Sección 2.3 queden documentados de forma verificable y este pendiente no se pierda del seguimiento.

REFERENCIAS

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., y Chen, W. (2022). LoRA: Low-rank adaptation of large language models. En *Proceedings of the International Conference on Learning Representations (ICLR)*.

Liu, X., Shen, S., Li, B., Ma, P., Jiang, R., Zhang, Y., Fan, J., Li, G., Tang, N., y Luo, Y. (2025a). NL2SQL-BUGs: A benchmark for detecting semantic errors in NL2SQL translation. En *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:2503.11984.

Liu, X., Shen, S., Li, B., Tang, N., y Luo, Y. (2025b). A survey of NL2SQL with large language models: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*. arXiv:2408.05109.

Pourreza, M., y Rafiei, D. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. En *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

Universidad Nacional de Colombia – Facultad de Ingeniería – Departamento de Sistemas e Industrial

Período 2026-15